

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
```

```
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names

print("X shape:", X.shape)
```

```
X shape: (150, 4)
```

```
mean_vec = np.mean(X, axis=0)
print("Mean of each feature:", mean_vec)
```

```
Mean of each feature: [5.84333333 3.05733333 3.758      1.19933333]
```

```
X_centered = X - mean_vec
print(np.round(np.mean(X_centered, axis=0), 8))
```

```
[-0. -0. -0. -0.]
```

```
N = X_centered.shape[0]
S = (X_centered.T @ X_centered) / (N - 1)
print(S)
```

```
[[ 0.68569351 -0.042434  1.27431544  0.51627069]
 [-0.042434   0.18997942 -0.32965638 -0.12163937]
 [ 1.27431544 -0.32965638  3.11627785  1.2956094 ]
 [ 0.51627069 -0.12163937  1.2956094   0.58100626]]
```

```
eigenvalues, eigenvectors = np.linalg.eigh(S)
print("Eigenvalues:", eigenvalues)
```

```
Eigenvalues: [0.02383509 0.0782095  0.24267075 4.22824171]
```

```
sort_idx = np.argsort(eigenvalues)[::-1]
eigenvalues_sorted = eigenvalues[sort_idx]
eigenvectors_sorted = eigenvectors[:, sort_idx]
```

```
explained_variance_ratio = eigenvalues_sorted / np.sum(eigenvalues_sorted)
cumulative_evr = np.cumsum(explained_variance_ratio)
```

```
for i, (evr, cum) in enumerate(zip(explained_variance_ratio, cumulative_evr), start=1):
    print(f"PC{i}: {evr*100:.2f}% | cumulative: {cum*100:.2f}%")
```

```
PC1: 92.46% | cumulative: 92.46%
PC2: 5.31% | cumulative: 97.77%
PC3: 1.71% | cumulative: 99.48%
PC4: 0.52% | cumulative: 100.00%
```

```
k = 2
W_k = eigenvectors_sorted[:, :k]
Z = X_centered @ W_k
print(Z.shape) # (150, 2)
```

```
(150, 2)
```

```
plt.figure(figsize=(7, 5))
colors = ['tab:red', 'tab:green', 'tab:blue']
for class_idx, class_name in enumerate(iris.target_names):
    plt.scatter(Z[y == class_idx, 0], Z[y == class_idx, 1],
                label=class_name, color=colors[class_idx], alpha=0.7)

plt.xlabel(f"PC1 ({explained_variance_ratio[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({explained_variance_ratio[1]*100:.1f}% variance)")
```

```
plt.title("Iris - PCA (manual)")  
plt.legend()  
plt.grid(alpha=0.3)  
plt.show()
```



